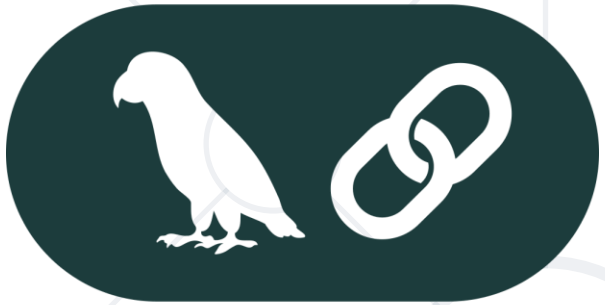
LangChain Memory & Human-in-the-Loop



SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

https://about.softuni.bg

Table of Contents

Context Engineering

Structuring the Model's Input

What is Context Engineering?

Software University

- **The Context Window**: LLMs have finite memory per request. Sending too much data increases latency, cost, and degrades reasoning.
- **Dynamic Injection**: Only give the model the exact information it needs for the *current* task.
- **Prompt Architecture**: Structuring system prompts, few-shot examples, and dynamic variables to guide the model safely.

```python
from langchain_core.prompts import PromptTemplate

# Keep templates focused and modular
prompt = PromptTemplate.from_template(
    "Given the user's role ({role}), answer this: {query}"
)
print(prompt.format(role="Admin", query="How to reset passwords?"))
```

Managing Chat History Context

Software University

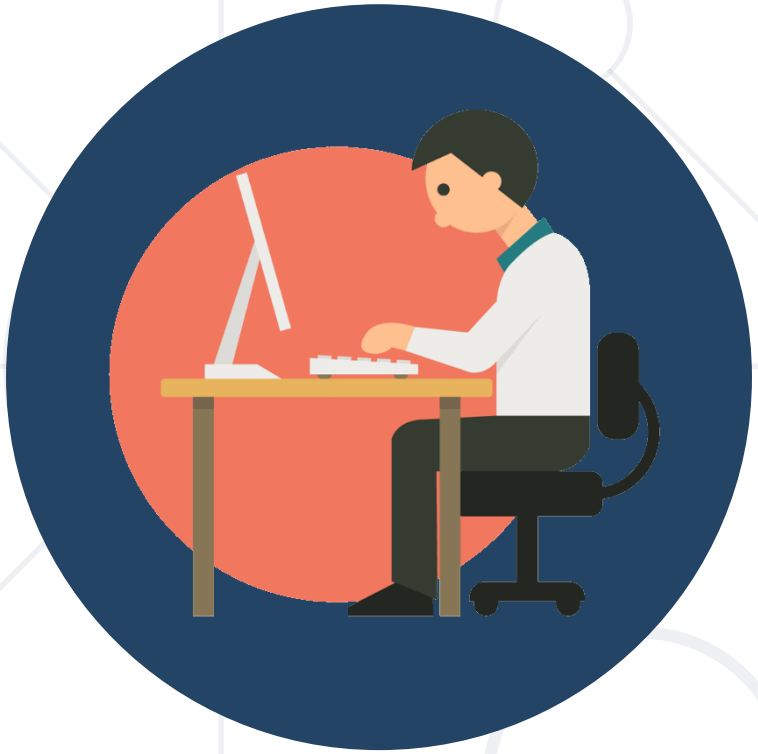
- **MessagesPlaceholder**: The standard way to inject dynamic lengths of conversation history into a prompt without hardcoding it.
- **Role-Based Formatting**: Ensuring user, AI, and system messages are clearly delineated for the model.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])

# Injecting the dynamic history list at runtime
formatted_prompt = prompt.invoke({
    "history": [HumanMessage("Hi"), AIMessage("Hello!")],
    "input": "What did I just say?"
})
```

Document Context (RAG Injection)

- **Formatting Retrieved Data**: Raw documents must be cleanly formatted into strings before the LLM can process them.
- **Semantic Density**: Passing only the most relevant snippets, not entire documents.

```
docs = [
    Document(page_content="LangChain is a framework."),
    Document(page_content="LangGraph adds state.")
]

# A simple formatting function used in context engineering pipelines
def format_docs(docs: list[Document]) -> str:
    return "\n\n".join(doc.page_content for doc in docs)
```



Practice

Exercise – Experiment with Context

Short-Term & Long-Term Memory

Managing Active Conversations and Cross-Session Knowledge

What is Short-Term Memory?

- **Thread Persistence:** Remembering the conversation history *within a single active session*.
- **AgentState:** The foundational LangGraph data structure that holds the current snapshot of the agent's context, memory, and intermediate steps.

Custom State & Reducers

- **Custom State Schema**: Define specific keys (e.g., user_id, plan, documents) using TypedDict or Pydantic.
- **State Reducers**: Functions that control how updates are applied (e.g., overwriting a value vs. appending to a list).
- **The Annotated Type**: Use Annotated[list, add_messages] to specify a reducer for a particular state key.
- **Flexibility**: Allows the agent to maintain complex internal logic beyond a simple message history.

Checkpointers

- **Freezing State:** Checkpointers save the entire graph state at every superstep.
- **In-Memory vs. Persistent:** Use InMemorySaver for testing, and Postgres/Sqlite savers for production.

```
from langgraph.checkpoint.memory import InMemorySaver
```

```
# 1. Initialize the checkpointer
memory = InMemorySaver()
```

```
memory_agent = create_agent(model=llm, checkpointer=memory)
```

Thread IDs

- **Resuming Conversations:** The agent's memory is retrieved using a unique thread_id stored inside the config object.
- **Multi-User Support:** By simply changing the thread_id, a single agent can manage thousands of independent, isolated conversations at once.

```
response_1 = memory_agent.invoke(
    {"messages": [HumanMessage(content="My name is Bob")]},
    config
)
print(response_1["messages"][-1].content)

response_2 = memory_agent.invoke(
    {"messages": [HumanMessage(content="What is my name?")]},
    config
)
print(response_2["messages"][-1].content)
```

```
config = {
    "configurable": {
        "thread_id": "user_123_session_1"
    }
}
```

Managing Memory Limits

- **The Problem**: Conversations get too long, exceeding the LLM's context window.
- **The Solution**: Using trim_messages to keep only the most recent N tokens or messages in the short-term memory before passing it to the LLM.
- Useful **Built-in Middlewares**: ContextEditingMiddleware, SummarizationMiddleware

Short-Term vs. Long-Term Memory

Software University

- **Short-Term (Thread Memory):** Isolated to a specific thread_id. Stores the exact sequence of messages for a single conversation.
- **Long-Term (User Memory):** Shared across multiple threads. Stores extracted facts, user preferences, and semantic knowledge.
- **The LangGraph Store:** A dedicated API specifically designed to handle this cross-thread, long-term persistence.

```python
from langgraph.store.memory import InMemoryStore

# The Store is entirely separate from the Checkpointer
store = InMemoryStore()
```

Namespaces and Key-Value Storage

Software University

- **Organization:** Memories are stored in a hierarchy (namespaces) like a file system.
- **Structured Data:** Memories are typically saved as dictionaries (JSON) rather than raw text.

```
namespace = ("users", "user_123", "preferences")

# Save a fact to long-term memory
store.put(namespace, "formatting", {"output_type": "markdown", "language": "bg"})

# Read it back in a completely different thread
memory = store.get(namespace, "formatting")
print("Retrieved Language:", memory.value["language"])
```

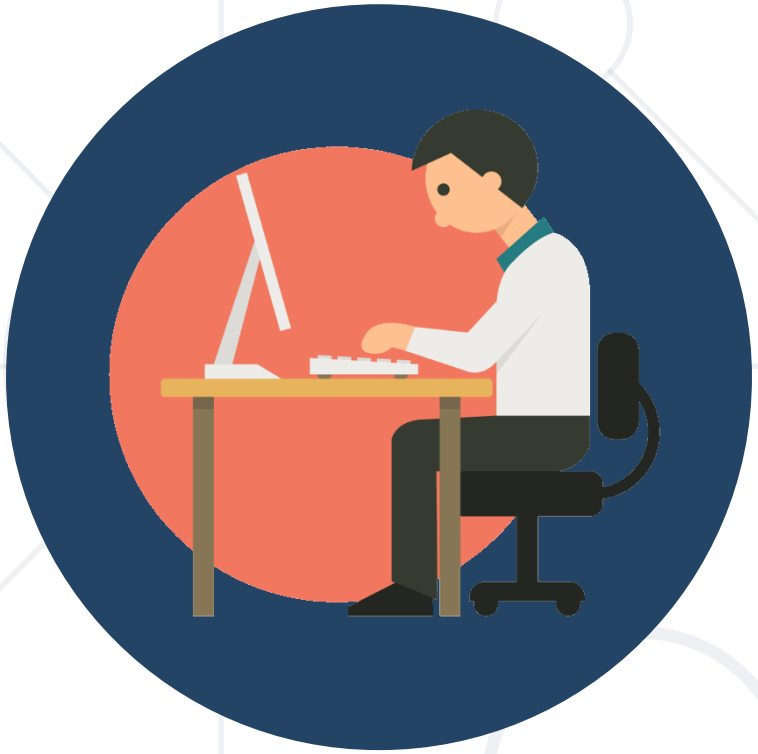
Accessing Store inside Nodes

- **Injected Dependency:** The Store is injected directly into your LangGraph nodes.
- **Read Before Act:** The agent reads long-term memory to condition its response before calling the LLM.

Profile Extraction (Updating Memory)

- **Background Tasks**: Agents can extract facts from the current conversation and write them back to the long-term store to remember for next time.

```python
def memory_middleware_node(state: MessagesState, config: RunnableConfig, store: BaseStore):
    # 1. Extract the unique user ID from the configuration
    user_id = config["configurable"].get("user_id", "unknown_user")

    # 2. Intercept the latest message
    last_message = state["messages"][-1].content.lower()

    # 3. Perform the middleware side-effect (saving to long-term memory)
    if "allergic to" in last_message:
        store.put(("users", user_id), "health", {"allergy": "peanuts"})

    # 4. Return the unmodified state to continue the graph execution
    return state
```



Practice

Exercise – Experiment with Memory

Guardrails & Human in the Loop

Ensuring Safe and Predictable Execution

Built-in Guardrails & PII Detection

- **Middleware Interception:** Guardrails build safe AI applications by validating and filtering content at key execution points using Middleware.
- **PII Detection:** LangChain provides built-in PIIMiddleware to handle Personally Identifiable Information (emails, credit cards, IPs).
- **Handling Strategies:** You can configure the middleware to handle detected PII via redact, mask, hash, or block strategies.

```
middleware=[
    PIIMiddleware(pii_type="email", strategy="redact", apply_to_input=True)
]
```

```
result_pii = safe_agent.invoke({
    "messages": [("user", "Check status for my email: professor_x@example.com")]
})

print(f"Agent received redacted input and responded: \n{result_pii['messages'][-1].content}")
```

```
Agent received redacted input and responded:
The status of the ticket for your email is active. If you need further assistance, feel free to ask!
```

Human-in-the-Loop (HITL)

Software University

- **High-Stakes Decisions**: The most effective guardrail for sensitive operations (e.g., deleting data, transferring money).
- **Built-in HumanInTheLoopMiddleware**: Automatically pauses execution and waits for a human command before sensitive tools are executed.
- **Auto-Approval**: You can strictly define which tools require interrupts and which can run autonomously.

```
middleware=[
    HumanInTheLoopMiddleware(
        interrupt_on={
            "delete_database_tool": True,
            "search_tool": False
        }
    )
],
```

Custom Guardrails (Before Agent)

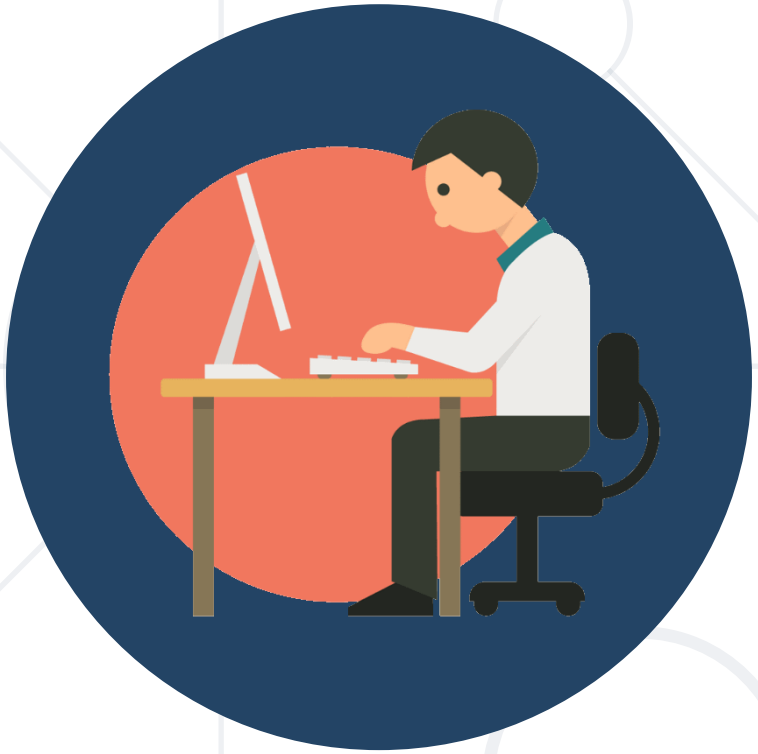
- **Deterministic Filtering:** Fast, cost-effective rule-based logic (regex, keyword matching).
- **Session-Level Checks:** Useful for checking authentication, rate limits, or blocking inappropriate requests *before* spending tokens.

Custom Guardrails & Layering

Software University

- **Model-Based Guardrails:** Use an LLM as a "judge" with semantic understanding to evaluate outputs.
- **Layering Guardrails:** You can safely combine multiple Middlewares in an array to create a robust, multi-layered security pipeline.

```
# Judge the safety of the output
check = self.judge.invoke(f"Is this text safe? Answer YES or NO: {last_msg.content}")
if "NO" in check.content.upper():
    last_msg.content = "I'm sorry, but I cannot provide that information for safety reasons."
return None
```



Practice

Exercise – Experiment with Guardrails

Observability & Evaluation

Tracing, Testing, and Improving

LangSmith Tracing & Observability

- **The Black Box Problem:** Complex agents execute unpredictable chains of tools and prompts. When they fail, finding the root cause is difficult.
- **Automatic Tracing:** LangSmith automatically logs every node, tool call, and LLM generation in your LangChain/LangGraph application.
- **Zero Code Changes:** Enabled purely via environment variables.

```
os.environ["LANGSMITH_TRACING"] =
os.environ["LANGSMITH_API_KEY"] =
os.environ["LANGSMITH_PROJECT"] =
```

Closing the Loop with User Feedback

Software University

- **Real-World Signals**: The best way to evaluate an LLM app is whether users actually find it helpful.
- **Run IDs**: Every LangChain execution generates a unique run_id.
- **Attaching Scores**: You can programmatically send feedback (thumbs up/down, or text corrections) back to the specific trace in LangSmith.

```python
from langsmith import Client

client = Client()

# Assuming your frontend captured a "thumbs up" for a specific generation
run_id = "a1b2c3d4-e5f6-7890-1234-567890abcdef"

# Attach the feedback to the trace
client.create_feedback(
    run_id=run_id,
    key="user_rating",
    score=1.0, # 1.0 for positive, 0.0 for negative
    comment="Agent used the perfect tool here."
)
```

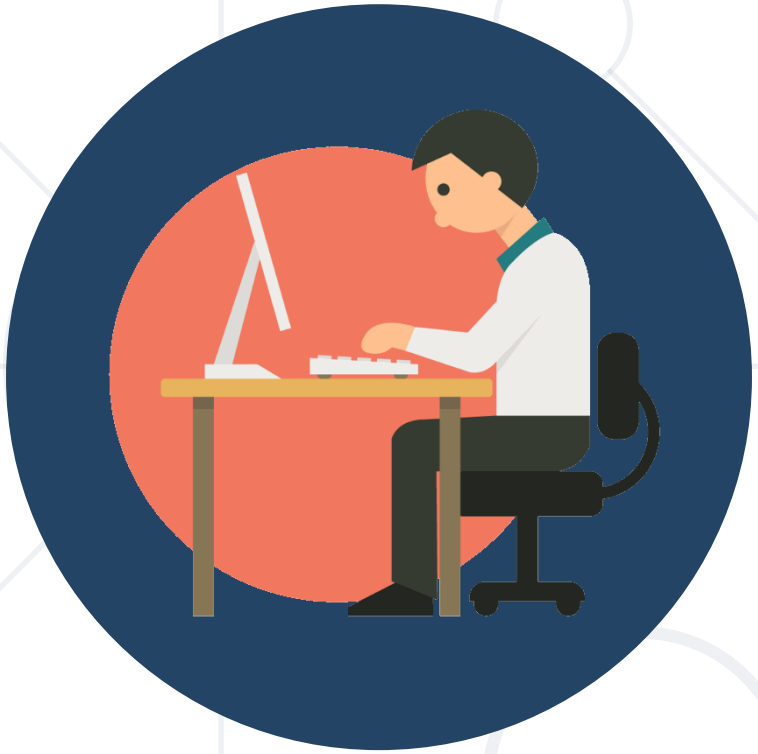
Creating Evaluation Datasets

- **Curating Golden Sets**: You can't improve what you don't measure. Datasets store pairs of "Inputs" and "Expected Outputs".
- **Production to Testing**: Easily convert interesting or failed production traces directly into dataset examples for future testing.
- **Version Control for Data**: Datasets are versioned, allowing you to track how your agent performs against an evolving set of user queries.

Automated Regression Tests

- **Testing Changes**: Before deploying a new prompt or switching to a cheaper model, you must ensure it doesn't break existing behavior.
- **Evaluators**: Functions that score the output. Can be simple heuristics (regex) or "LLM-as-a-Judge".
- **The evaluate API**: LangSmith runs your agent over the entire dataset concurrently and generates a detailed performance report.

```
✅ Regression Test Complete! Check your LangSmith dashboard for the UI visualization.
```



Practice

Exercise – Experiment with LangSmith

Summary

- Use **LangGraph** to manage complex state and dynamic **context** windows for more reliable agent reasoning.
- Integrate session-based **checkpointers** and long-term storage to maintain continuity across every user interaction.
- Implement hard **guardrails** and **human-in-the-loop** interrupts to ensure safety and oversight for high-stakes actions.
- Transition from black-box experiments to production-grade systems using **LangSmith** tracing and automated **evaluation**.